

Motivación

Un sistema de software se desarrolla con el objetivo de resolver un problema que puede surgir de una necesidad, una oportunidad o una idea. El desarrollo de software es un proceso de abstracción que permite construir un modelo para la solución de un problema.

La construcción de un modelo implica un esfuerzo intelectual, cuya complejidad depende de las características del problema y por supuesto del conocimiento del equipo de desarrollo responsable de hallar una solución. A esa complejidad se suma la necesidad de escribir código utilizando un lenguaje de programación.

Sin embargo, escribir código es una tarea relativamente sencilla respecto al resto de las etapas involucradas en el proceso. Especificar los requerimientos del problema y diseñar una solución, son etapas previas a escribir el código y por lo general son las que demandan mayor nivel de abstracción. Verificar que el código es una solución para el problema especificado por los requerimientos, es una etapa posterior, que también implica esfuerzo y conocimiento. Desarrollar sistemas de software de calidad para ambientes dinámicos, requiere la aplicación de principios y metodologías que guíen cada etapa del proceso.

En este contexto, enseñar a desarrollar software o, como contrapartida, aprender a desarrollar software, no es una tarea sencilla. Aun en los niveles iniciales, demanda un enfoque que de manera sistemática y disciplinada introduzca principios y metodologías adecuadas para construir sistemas de mediana y gran escala, que satisfagan factores de calidad.

La programación orientada a objetos brinda un paradigma que guía el proceso de desarrollo de software desde su concepción. En la actualidad se aplica tanto en la producción de aplicaciones de pequeña escala, como para la construcción de sistemas complejos. Sin embargo, no es sencillo ilustrar la potencia del paradigma y sobre todo enfatizar la importancia de adoptar un estilo disciplinado y sistemático, planteando problemas de pequeña escala.

El objetivo de este libro es introducir los conceptos fundamentales del paradigma orientado a objetos y desarrollar competencias para interpretar un modelo orientado a objetos, implementarlo en Java y verificarlo para un conjunto de casos de prueba. Nuestra propuesta es presentar estos conceptos y desarrollar estas competencias a partir de distintos problemas especificados a través de un texto y un diagrama de clases. Así, las etapas de desarrollo de requerimientos y diseño se asumen realizadas previamente.

Los contenidos conceptuales se introducen de manera transversal a la presentación del lenguaje de programación. Esto es, para cada uno de los conceptos abordados, se plantean problemas en cuya resolución se aplican estos conceptos, se describen las facilidades provistas por Java para soportarlos y se analiza la calidad de la solución en términos de legibilidad, eficiencia y correctitud.

En esta propuesta, el lenguaje de programación es una herramienta que se irá presentando a través de la resolución de problemas concretos, sin una descripción exhaustiva de cada uno de los recursos. Se asume que los estudiantes han adquirido previamente conocimientos y habilidades para:

- Interpretar enunciados de problemas incluyendo definiciones usando notación matemática.
- Diseñar algoritmos aplicando los lineamientos del diseño top down y el refinamiento paso a paso.
- Implementar soluciones en un lenguaje de programación utilizando adecuadamente las estructuras de control a propuestas por la programación estructurada.
- Verificar la solución implementada en lenguaje de programación respecto al enunciado, para un conjunto significativo de casos de prueba.
- Documentar el código para favorecer la legibilidad.

A través de la resolución de los problemas planteados se aspira a desarrollar los siguientes resultados del aprendizaje:

- Interpretar un diagrama de clases que incluye relaciones de asociación, dependencia y herencia.
- Implementar en Java un diagrama de clases considerando la funcionalidad y las responsabilidades especificadas.
- Diseñar algoritmos eficientes para modelar servicios que permitan procesar y actualizar un arreglo de componentes.
- Verificar la implementación de un método, clase o colección de clases, respecto a su especificación, utilizando casos de prueba adecuados.
- Comprender los conceptos de encapsulamiento, herencia y polimorfismo y su impacto sobre la calidad y productividad del software.

La expectativa es que el conocimiento y las habilidades desarrolladas sirvan de base para profundizar en el aprendizaje de temas de programación e ingeniería de software.

El Proceso de Desarrollo de Software

Los sistemas de software actuales suelen resolver problemas complejos que requieren soluciones confiables, eficientes y capaces de adaptarse dinámicamente a cambios en las necesidades de los **clientes** o **usuarios**. El desarrollo de un sistema de software de estas características es un **proceso** que tiene un **ciclo de vida** conformado por etapas que pueden organizarse de diferentes formas. El **producto** final de este proceso es un sistema de software.

El proceso se realiza en el marco de un **proyecto** que establece un **cronograma** y un **presupuesto** para la elaboración de un **producto**. El producto es justamente un sistema de software. El éxito del proyecto está ligado a la **calidad** del **producto**, pero también es fundamental que se complete con los **costos** previstos en el presupuesto y los **tiempos** establecidos en el cronograma.

Los proyectos de desarrollo de software puede clasificarse de acuerdo a diferentes criterios. En este libro se proponen tres categorías, sistemas de **pequeña, mediana y gran escala**. Aunque no hay límites precisos que separen a estas categorías, el tiempo y el costo de desarrollo son los principales factores que determinan la escala de un sistema.

En los proyectos de mediana y gran escala participa un **equipo de desarrollo** conformado por profesionales con diferentes capacidades, cada uno de los cuales tiene un rol protagónico en alguna de las etapas del proceso. Uno de los recursos con mayor incidencia en el costo de un proyecto es justamente el equipo de desarrollo.

Aunque el desarrollo de software es una actividad creativa, requiere de la aplicación de un **paradigma** que guíe, oriente y sistematice cada **etapa**. Un paradigma de programación brinda:

- **Un principio** que describe propiedades generales que se aplican a todo el proceso de desarrollo.
- **Una metodología** que consta de un conjunto integrado de métodos, estrategias y técnicas que aseguran la aplicación del principio.
- **Un conjunto de herramientas** que soportan y facilitan la aplicación de la metodología.

El paradigma de programación orientada a objetos, se aplica tanto en proyectos de pequeña o mediana escala, como en el desarrollo de grandes sistemas. Dos de las herramientas fundamentales del proceso de desarrollo de software, son el **lenguaje de modelado** y el **lenguaje de programación**.

Ciclo de vida

El ciclo de vida de un sistema de software comienza cuando se identifica el problema o se concibe la idea que le da origen y termina cuando el producto deja de utilizarse. Un modelo de ciclo de vida propone un enfoque específico para establecer y ordenar las etapas que conforman el proceso de desarrollo de un producto de software.

En los modelos de ciclo de vida **secuenciales** las etapas se ordenan en forma lineal. El **resultado** de una etapa es el **insumo** para la siguiente hasta alcanzar el producto final. En los modelos de ciclo de vida **evolutivos** se desarrolla una versión inicial del producto que luego se va refinando al incorporar nuevas funcionalidades o al mejorar los atributos de calidad. El proceso también se organiza en etapas pero pueden solaparse en el tiempo. En los modelos evolutivos las primeras versiones se entregan sin conocer los alcances del producto final.

Cualquiera sea el modelo de ciclo de vida, en el proceso intervienen diferentes participantes, entre ellos el cliente que demanda el producto, el equipo de profesionales que lo desarrolla y los usuarios que en definitiva van a interactuar con el sistema.

La **documentación** es una etapa transversal a todo el proceso. En cada etapa se produce un documento que sirve de insumo para las que siguen. El software está formado por el código escrito en un lenguaje de programación y el conjunto de documentos que se generan durante el proceso de desarrollo.

Con diferentes nombre dependiendo del modelo de ciclo de vida, las etapas del proceso de desarrollo de software son:

Desarrollo de los Requerimientos

Un **sistema de software** se desarrolla para resolver un **problema** que puede surgir de una **necesidad**, una **oportunidad** o una **idea**. Un proyecto será exitoso si el sistema es una solución para el problema, para ello deben definirse con precisión los **requerimientos**.

El **resultado** de esta etapa especifica:

- **Qué problema** tiene que ser resuelto.
- **Por qué** es un problema y por lo tanto requiere solución.

- **Qué** cualidades debe exhibir la solución, **qué** funcionalidades debe brindar y **qué** restricciones debe cumplir.
- **Quiénes** tienen la responsabilidad de participar en la construcción de la solución. Un problema puede surgir dentro de una organización que desarrolla un sistema para uso propio, ya sea a partir de una demanda del entorno interno o del contexto externo. Alternativamente el problema puede surgir dentro de una organización para crear y comercializar un sistema de software en su contexto externo.

Diseño

A partir de los requerimientos se **diseña** una **solución** para el problema especificado. El **resultado** de esta etapa es una especificación de los módulos que integrarán el sistema y el modo en que se relacionan entre sí. La especificación puede establecer también **casos de prueba** o **tests** que se aplicarán en la verificación.

Implementación

A partir de la especificación producida durante la etapa de diseño, los **desarrolladores** o **programadores** generan el **programa** escrito en un lenguaje de programación y toda la documentación referida al código. Es importante que el programa implementado mantenga la estructura de la solución especificada en la etapa de diseño. Cada **módulo de diseño** debería corresponderse con una **unidad de código** implementada, con cierta independencia del sistema completo.

Verificación y depuración

La verificación evalúa si el sistema satisface la especificación de requerimientos. Durante la depuración se corrigen los errores que se detectan. Cuando el sistema está dividido en módulos, cada unidad de código se verifica y depura por separado y luego se verifica y depura la integración de los módulos. Es importante que al menos una parte de la verificación la lleven a cabo personas ajenas a la implementación. Los casos de prueba pueden haberse establecido en el diseño o pueden ser definidos en esta misma etapa.

El término validación se utiliza para referirse a la evaluación de un sistema respecto a las necesidades reales del usuario, que en ocasiones no corresponden a los requerimientos especificados.

Considerando que el principal objetivo de este libro es introducir conceptos de programación orientada a objetos, no se distingue entre validación y verificación, asumiendo que los requerimientos de los problemas propuestos, corresponden a las necesidades de los usuarios que los plantean.

El lenguaje de modelado comienza a utilizarse durante el desarrollo de requerimientos y permite elaborar diferentes tipos de **diagramas**. El lenguaje de programación afecta fundamentalmente a la etapa de implementación.

Durante el ciclo de vida de un sistema de software las necesidades del usuario cambian y normalmente crecen. El diseño modular es fundamental para controlar el impacto de los cambios. En un sistema bien modulado los cambios menores impactan sobre un conjunto reducido de módulos o incluso pueden llegar a provocar la necesidad de agregar nuevos módulos, sin afectar a los que ya están implementados y verificados.

En los modelos de ciclo de vida secuenciales la etapa de mantenimiento involucra todos los cambios en el software que resultan de modificaciones en la especificación de requerimientos o en diseño. En los modelos de ciclo de vida evolutivos no se plantea una etapa para el mantenimiento sino que el proceso de desarrollo se organiza anticipándose a los cambios.

El paradigma de programación orientada a objetos y los modelos evolutivos son particularmente adecuados para ambientes dinámicos con altos requerimientos de calidad y productividad.

Calidad

En un sentido estricto la calidad de un sistema de software se evalúa considerando el nivel de satisfacción que alcanza el usuario o cliente a partir del momento que comienza a utilizarlo. Un mecanismo menos exigente evalúa la calidad del sistema con relación a los requerimientos acordados.

En cualquier caso la calidad se evalúa a partir de diferentes **atributos o factores**, entre ellos:

Correctitud: Un producto de software es correcto si actúa de acuerdo a los requerimientos especificados.

Eficiencia: Un producto de software es eficiente si tiene una baja demanda de recursos de hardware, en particular tiempo de CPU, espacio de memoria y ancho de banda.

Portabilidad: Un producto de software es portable si puede ejecutarse sobre diferentes plataformas de hardware y de software.

Simplicidad: Un producto de software es simple si es fácil de usar, su interfaz es amigable y no requiere demasiado entrenamiento ni capacitación por parte del usuario.

Robustez: Un producto de software es robusto si reacciona adecuadamente aun en circunstancias imprevisibles.

Disponibilidad: Un producto de software tiene buena disponibilidad si puede ser usado en el momento que el usuario lo necesita y el rendimiento está dentro de parámetros establecidos.

Legibilidad: Un producto de software es legible si un desarrollador o incluso otros miembros del equipo de desarrollo, puede leerlo e interpretar su estructura y contenido fácilmente.

Los factores de calidad son temas esenciales de la ingeniería de software, este libro se concentra en aquellos que son fundamentales en las etapas formativas de un programador y por lo tanto se utilizarán para evaluar las soluciones de los problemas propuestos.

Productividad

En los últimos años se ha destinado mucho esfuerzo a desarrollar lenguajes y metodologías que permitan aumentar la **productividad** del software. La productividad está ligada al **costo** y al **tiempo** que demanda la concepción y construcción de un sistema e incide en cada etapa de su desarrollo.

Cualquiera sea el modelo de proceso, si una etapa del desarrollo de un producto de software se saltea o no se completa adecuadamente, las etapas siguientes sufrirán las consecuencias, el proyecto probablemente termine demandando más tiempo y el costo global será mayor al presupuestado. Por el contrario, si cada etapa se realiza de manera adecuada, el proyecto se completará de acuerdo al cronograma y al presupuesto establecido. Esto no garantiza el éxito,

existen otros aspectos, en general externos al proyecto, que inciden en la aceptación del producto.

La productividad está vinculada a dos factores fundamentales, **extensibilidad** y **reusabilidad**.

Extensibilidad: Un producto de software es extensible si el costo y el tiempo que demanda un cambio en la especificación, es consistente con la envergadura del cambio.

Reusabilidad: Un módulo de software o una colección de módulos es reusable si puede utilizarse para la construcción de diferentes aplicaciones.

La extensibilidad afecta fundamentalmente al mantenimiento, pero depende de las etapas anteriores. Los cambios en un sistema con un diseño flexible tendrán un costo menor que los que provocará la extensión de una solución con un diseño pobre.

La reusabilidad afecta a todas las etapas. Cuando el equipo de desarrollo resuelve un problema puede reusar requerimientos, módulos de diseño o código de sistemas anteriores. Como contrapartida, construir un módulo para una aplicación, considerando que en el futuro sea reutilizado, tiene un costo mayor de modo que resulta una ventaja si efectivamente vuelve a usarse.

Ambas cualidades están ligadas a la **legibilidad** un factor de calidad que el usuario no percibe, pero que sí es valioso para el equipo de desarrollo. La documentación y estructuración del código favorecen la extensión y el reuso. Adoptar algunas convenciones para la indentación y la selección de los identificadores también favorece la legibilidad y contribuye a la productividad.

Programación orientada a objetos

El **principio** fundamental del **paradigma de programación orientada a objetos** es desarrollar un sistema de software en base a las entidades relevantes del problema que le da origen.

Una **metodología orientada a objetos** parte del reconocimiento de los **objetos del problema**. Cada objeto del problema es una entidad que puede caracterizarse a través de sus **atributos** y su **comportamiento**. Los atributos y el comportamiento permiten agrupar a los objetos en **clases**. Una metodología orientada a objetos abarca el ciclo de vida completo de un producto de software y brinda herramientas adecuadas para cada etapa del proceso.

El **conjunto de herramientas** incluye a un **lenguaje de modelado** y un **lenguaje de programación**. Un lenguaje de modelado es una notación artificial que permite describir la estructura y las componentes de un sistema de software. Un lenguaje de modelado consistente con la programación orientada a objetos, permite representar a la colección de clases y sus relaciones a través de un **diagrama de clases**. Un lenguaje de programación orientado a objetos brinda mecanismos que facilitan la implementación del sistema diseñado.

En la etapa de **desarrollo de requerimientos** la especificación de:

- **Qué** problema tiene que ser resuelto.
- **Qué** cualidades debe exhibir la solución, **qué** funcionalidades debe brindar y **qué** restricciones debe cumplir.

se aborda a partir de la identificación de los objetos del problema y su clasificación. Una **clase** define los atributos que caracterizan a un conjunto de objetos. Las clases se vinculan entre sí

a través de diferentes mecanismos de relación. El documento producido en esta etapa incluye un **diagrama de clases** que modela la colección de clases y sus relaciones.

En la etapa de **diseño** se establece el comportamiento de los objetos y se completa la especificación de las clases y relaciones.

En la actualidad UML es el lenguaje de modelado más utilizado cuando se aplica la programación orientada a objetos. UML permite elaborar diferentes tipos de diagramas, en particular diagramas de clases.

Existen numerosos lenguajes de programación orientados a objetos que facilitan la implementación del sistema diseñado. Java es un lenguaje que adopta los lineamientos de la programación orientada a objetos y se utiliza tanto en el ámbito académico como comercial.

Es importante considerar que una metodología favorece el proceso de desarrollo, pero no es una receta infalible y completa que garantice la calidad y productividad. Los lenguajes de modelado y los lenguajes de programación orientados a objetos brindan mecanismos consistentes con la metodología, pero no garantizan que la implementación sea una representación alienada con el diseño.

En los casos de estudio y ejercicios propuestos en los capítulos que siguen se asume que ya se han especificado los requerimientos y se ha diseñado un modelo que debe ser implementado en un lenguaje de programación. Los enunciados incluyen un texto y un diagrama de clases que en conjunto modelan el problema a resolver. Se emplea una versión simplificada de UML para los diagramas de clases y Java como lenguaje de programación. Se explican las principales características de Java, asumiendo cierto nivel de autonomía para el aprendizaje de los aspectos básicos de un lenguaje de programación.